

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN, ĐHQG-HCM
KHOA MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG



BÁO CÁO ĐỒ ÁN MÔN HỌC

**ĐỀ TÀI: Hệ thống an ninh thông minh sử dụng
ESP32-CAM và AI nhận diện**

Môn học: NT131.Q24 - Hệ Thống Nhúng Mạng Không Dây

Giảng viên hướng dẫn: Lê Minh Khánh Hội

Thực hiện bởi nhóm 5, bao gồm:

Họ và tên	MSSV	Chức vụ
Lâm Mai Hồ Nhật Khánh	24520782	Trưởng Nhóm
Nguyễn Văn Phát	24521312	Thành Viên
Dương Quốc Trung	24521877	Thành Viên
Mai Đăng Khoa	24520817	Thành Viên

Thời gian thực hiện: 1/4/2026 - 1/6/2026

Lời Cảm Ơn

Lời đầu tiên, chúng em xin gửi lời cảm ơn chân thành và sâu sắc đến cô Lê Minh Khánh Hội – giảng viên môn Hệ thống nhúng và mạng không dây - Trường Đại Học Công Nghệ Thông Tin - ĐHQG TP HCM người đã tận tình hướng dẫn, hỗ trợ và truyền đạt cho chúng em những kiến thức quý báu trong suốt quá trình thực hiện đồ án.

Trong suốt thời gian học tập và nghiên cứu, cô không chỉ giúp chúng em hiểu rõ hơn về các kiến thức chuyên môn mà còn luôn động viên, góp ý để chúng em hoàn thiện đề tài một cách tốt nhất. Những nhận xét chi tiết cùng sự tận tâm của cô đã giúp chúng em khắc phục nhiều khó khăn và nâng cao kỹ năng của bản thân.

Chúng em xin chân thành cảm ơn cô vì sự nhiệt tình, tâm huyết và trách nhiệm trong công tác giảng dạy. Đây sẽ là hành trang quý giá giúp chúng em tự tin hơn trong con đường học tập và công việc sau này.

Một lần nữa, chúng em xin kính chúc cô luôn mạnh khỏe, hạnh phúc và tiếp tục gặt hái nhiều thành công trong sự nghiệp giảng dạy. Chúng em xin chân thành cảm ơn !

Nhận Xét

[illegible]

MỤC LỤC

MỤC LỤC.....	4
MỤC LỤC HÌNH ẢNH.....	6
PHẦN I. TỔNG QUAN.....	7
1.1 Giới Thiệu Và Lý Do Chọn Đề Tài.....	7
1.2 Đối tượng và phương pháp nghiên cứu.....	8
1.2.1 Đối tượng nghiên cứu.....	8
1.2.2 Phương pháp nghiên cứu.....	8
1.3 Phân công công việc.....	9
PHẦN II. CƠ SỞ LÝ THUYẾT.....	10
2.1. Giới Thiệu Linh Kiện Điện Tử.....	10
2.1.1. Module Wifi Camera ESP32.....	10
2.1.2 Module còi chip 3.3V-5V.....	10
2.1.3 Module cảm biến PIR HC-SR501.....	11
2.1.4 Board mạch cắm mini 400 Lỗ.....	12
2.1.5 Dây cắm truyền dữ liệu đực - cái và đực - đực.....	12
2.1.6 Servo motor.....	13
2.2. Hệ điều hành Ubuntu.....	13
2.3. Các phần mềm hỗ trợ.....	14
2.3.1 MQTTx.....	14
2.3.2 Arduino IDE 2.x.....	14
2.3.3 Python 3.11+.....	14
2.3.4 Telegram BOT.....	15
2.3.5 Git Và GitHub.....	15
2.3.6 AWS Rekognition.....	15
2.4. Các giao thức.....	16
2.4.1. MQTT (Pub/Sub).....	16
2.4.2 HTTP ,HTTPS , NTP.....	17
2.5. Đề xuất giải pháp.....	19
PHẦN III. THIẾT KẾ VÀ PHÂN TÍCH HỆ THỐNG.....	19
3.1 Thiết kế hệ thống.....	19
3.1.1 Danh sách thiết bị và chi phí.....	19
3.1.2 Sơ đồ kiến trúc hệ thống.....	21
3.1.3 Sơ đồ mạch.....	23
3.1.4 Sơ đồ hoạt động.....	25
3.1.5 Cấu hình MQTT Broker (HiveMQ Cloud).....	27
3.2 Phân tích chức năng và khởi động hệ thống.....	30
3.2.1 Phân tích chức năng.....	30
3.2.2 Khởi động hệ thống.....	35
PHẦN IV. XÂY DỰNG THỬ NGHIỆM VÀ ĐÁNH GIÁ.....	37
4.1 Môi trường thử nghiệm thực tế.....	37
4.2 Kết quả thử nghiệm và đánh giá.....	38

4.2.1 Kết quả vận hành tổng thể.....	38
4.2.2 Độ chính xác AI nhận diện.....	39
4.2.3 Kết quả gửi cảnh báo Telegram.....	39
4.2.4 Hiệu suất và độ trễ xử lý.....	41
4.2.5 Đánh giá tổng thể.....	42
PHẦN V. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	42
5.1 Kết luận.....	42
5.2 Hướng phát triển trong tương lai.....	42
THAM KHẢO.....	44

MỤC LỤC HÌNH ẢNH

Hình 1: Module ESP32-CAM.....	10
Hình 2: Module còi.....	11
Hình 3: Cảm biến PIR HC-SR501.....	11
Hình 4: Breadboard mini.....	12
Hình 5: Dây cắm truyền dữ liệu.....	13
Hình 6: Servo motor.....	13
Hình 7: Bảng chi phí các linh kiện sử dụng.....	20
Hình 8: Sơ đồ kiến trúc hệ thống.....	22
Hình 9: Sơ đồ mạch.....	23
Hình 10: Sơ đồ luồng hoạt động.....	25
Hình 11: Thông số kết nối HiveMQ Cloud cluster.....	28
Hình 12: Cấu hình MQTT broker trong config.py.....	29
Hình 13: Danh sách credentials được cấu hình trên HiveMQ Cloud.....	29
Hình 14: Thông điệp json ESP32-CAM gửi đến topic tương ứng.....	30
Hình 15: Cấu trúc file hệ thống.....	31
Hình 16: Cấu hình AI Detection và AWS Rekognition trong config.py.....	33
Hình 17: Nội dung file known_faces.json.....	34
Hình 18: Lệnh cài đặt dependencies.....	36
Hình 19: Lệnh kích hoạt môi trường ảo.....	36
Hình 20: Backend Flask nhận và xử lý dữ liệu từ ESP32-CAM.....	37
Hình 21: Kết quả gửi qua Telegram khi nhận diện người quen.....	40
Hình 22: Kết quả gửi qua Telegram khi nhận diện người lạ.....	41

PHẦN I. TỔNG QUAN

1.1 Giới Thiệu Và Lý Do Chọn Đề Tài

Trong kỷ nguyên công nghệ số, vấn đề an ninh và giám sát không gian sống, làm việc ngày càng trở nên cấp thiết do sự gia tăng các mối đe dọa từ bên ngoài. Đồ án "Hệ thống bảo mật thông minh sử dụng ESP32-CAM và trí tuệ nhân tạo phát hiện" được thực hiện nhằm xây dựng một giải pháp toàn diện, tích hợp phần cứng IoT với công nghệ AI để tự động phát hiện và cảnh báo các sự kiện bất thường. Hệ thống sử dụng camera ESP32-CAM để thu thập dữ liệu hình ảnh thời gian thực, kết hợp với mô hình AI (như AWS Rekognition hoặc các thuật toán phát hiện khuôn mặt tự xây dựng) để phân tích và nhận diện đối tượng. Bên cạnh đó, backend server (dựa trên Python Flask) xử lý dữ liệu, giao thức MQTT đảm bảo truyền tải thông tin ổn định, và dịch vụ Telegram cung cấp cảnh báo tức thì cho người dùng.

Đồ án tập trung vào việc thiết kế, triển khai và thử nghiệm hệ thống, bao gồm các module chính như thu thập dữ liệu, xử lý AI, điều phối backend và giao tiếp người dùng. Mục tiêu chính là tạo ra một sản phẩm có khả năng mở rộng, chi phí thấp và hiệu quả cao, góp phần nâng cao chất lượng an ninh trong các ứng dụng thực tế như bảo vệ gia đình, văn phòng hoặc khu công nghiệp.

Việc lựa chọn đề tài "Hệ thống bảo mật thông minh sử dụng ESP32-CAM và trí tuệ nhân tạo phát hiện" xuất phát từ những lý do sau:

Nhu cầu xã hội và tính thời sự: An ninh là vấn đề toàn cầu, đặc biệt trong bối cảnh đô thị hóa và số hóa. Các hệ thống giám sát truyền thống thường thiếu tính tự động và chính xác, dẫn đến nhiều rủi ro. Đề tài này giải quyết vấn đề thực tế bằng cách ứng dụng IoT và AI, giúp giảm thiểu tội phạm và tăng cường sự an toàn cộng đồng.

Giá trị học thuật và kỹ thuật: Đề tài cho phép nghiên cứu sâu về các lĩnh vực công nghệ tiên tiến như lập trình nhúng (ESP32), xử lý hình ảnh AI, phát triển backend. Điều này giúp rèn luyện kỹ năng đa ngành, từ phần cứng đến phần mềm, phù hợp với xu hướng phát triển của ngành Công nghệ Thông tin.

Tiềm năng ứng dụng và kinh tế: Hệ thống có thể triển khai dễ dàng với chi phí thấp, mang lại lợi ích kinh tế thông qua việc thương mại hóa hoặc mở rộng thành sản phẩm. Đồng thời, nó đáp ứng nhu cầu của thị trường IoT đang bùng nổ, tạo cơ hội việc làm và đổi mới công nghệ.

Là một sinh viên đam mê công nghệ, chúng em muốn khám phá cách thức kết hợp AI với IoT để tạo ra các giải pháp thông minh. Đề tài này không chỉ thỏa mãn đam mê mà còn giúp chúng em đóng góp vào việc cải thiện an ninh xã hội thông qua nghiên cứu thực tiễn.

1.2 Đối tượng và phương pháp nghiên cứu

1.2.1 Đối tượng nghiên cứu

Đề án "Hệ thống bảo mật thông minh sử dụng ESP32-CAM và trí tuệ nhân tạo phát hiện" tập trung nghiên cứu và phát triển một hệ thống IoT toàn diện để giám sát và cảnh báo an ninh. Các đối tượng nghiên cứu chính bao gồm:

Phần cứng ESP32-CAM: Các module camera ESP32-CAM (cam 1 và cam 2) tích hợp cảm biến PIR để phát hiện chuyển động, buzzer và đèn flash LED để phản hồi cục bộ, cùng với khả năng chụp và truyền ảnh qua MQTT.

Công nghệ AI phát hiện khuôn mặt: Mô hình nhận diện khuôn mặt sử dụng AWS Rekognition (thông qua thư viện boto3), bao gồm phát hiện khuôn mặt, so khớp với cơ sở dữ liệu đã biết (known_faces.json), và lưu trữ khuôn mặt lạ (unknown_faces/).

Backend xử lý dữ liệu: Server Flask (app.py) để nhận dữ liệu từ MQTT, xử lý hình ảnh bằng Pillow, và điều phối các module AI và cảnh báo. Giao thức truyền tải: MQTT (sử dụng paho-mqtt) để kết nối ESP32 với backend, đảm bảo truyền tải thời gian thực và ổn định. Dịch vụ cảnh báo: Tích hợp Telegram Bot (telegram_alerts.py và services/telegram.py) để gửi thông báo tức thì về phát hiện chuyển động hoặc khuôn mặt lạ. Điều phối đa camera: Module MultiCameraCoordinator để quản lý nhiều camera ESP32, tránh trùng lặp cảnh báo và tối ưu hóa hiệu suất.

Người dùng và môi trường ứng dụng: Các cá nhân hoặc tổ chức sử dụng hệ thống để bảo vệ không gian sống/làm việc, với trọng tâm vào độ chính xác phát hiện (ngưỡng 99% confidence), thời gian phản hồi và khả năng mở rộng.

Đối tượng nghiên cứu không chỉ dừng lại ở kỹ thuật mà còn bao gồm hiệu quả thực tế, độ tin cậy trong môi trường thực tế và khả năng tích hợp với các dịch vụ AWS.

1.2.2 Phương pháp nghiên cứu

Phương pháp nghiên cứu được thực hiện theo quy trình phát triển phần mềm và kỹ thuật IoT, dựa trên kiến trúc hệ thống đã triển khai, bao gồm các bước sau:

Nghiên cứu lý thuyết: Thu thập và phân tích tài liệu về ESP32-CAM (lập trình nhúng với Arduino IDE), AI nhận diện khuôn mặt (AWS Rekognition API, boto3), giao thức MQTT (PubSubClient), và phát triển backend (Flask, requests). Tham khảo tài liệu kỹ thuật từ AWS, ESP32 documentation và các bài báo về IoT bảo mật.

Thiết kế hệ thống: Sử dụng mô hình hóa kiến trúc tổng thể với sơ đồ luồng dữ liệu (PIR → ESP32 → MQTT → Backend → AI → Telegram), thiết kế cấu trúc thư mục (backend/, esp32/, ai/,) và định nghĩa cấu hình (config.py) cho các tham số như ngưỡng phát hiện, MQTT broker (HiveMQ cloud), và Telegram bot.

1.3 Phân công công việc

Họ và tên	MSSV	Nhiệm vụ	Mức độ hoàn thành
Lâm Mai Hồ Nhật Khánh	24520782	Trưởng nhóm. Thiết kế kiến trúc hệ thống, lập trình backend Flask, tích hợp MQTT client, cấu hình HiveMQ Cloud, viết báo cáo tổng hợp	100%
Nguyễn Văn Phát	24521312	Lập trình firmware ESP32-CAM (cam1, cam2), kết nối cảm biến PIR, buzzer, servo, kiểm thử phần cứng, tích hợp Telegram Bot	100%
Dương Quốc Trung	24521877	Hỗ trợ xây dựng nội dung báo cáo, thiết kế slide thuyết trình, hỗ trợ quay video demo và kiểm thử hệ thống	100%
Mai Đăng Khoa	24520817	Tích hợp AWS Rekognition, xây dựng module AI detection, quản lý known_faces.json, kiểm thử nhận diện	100%

PHẦN II. CƠ SỞ LÝ THUYẾT

2.1. Giới Thiệu Linh Kiện Điện Tử

2.1.1. Module Wifi Camera ESP32



Hình 1: Module ESP32-CAM

ESP32-CAM AI-Thinker là module siêu nhỏ công suất thấp có hai CPU LX6 32bit hiệu suất cao với kiến trúc vi mạch đường dẫn 7 lớp. ESP32-CAM được tích hợp với Wi-Fi, Bluetooth và có thể sử dụng với máy ảnh OV2640 hoặc OV7670. IC ESP32 có các giao thức ADC, SPI, I2C và UART độ phân giải cao để giao tiếp dữ liệu. Module có cảm biến Hall, cảm biến nhiệt độ và cảm biến cảm ứng, và bộ hẹn giờ watchdog. Module ESP32-CAM AI-Thinker có nhiều ứng dụng như tự động hóa trong nhà, thiết bị thông minh, hệ thống định vị, hệ thống an ninh và thích hợp cho các ứng dụng IoT. [1]

2.1.2 Module còi chip 3.3V-5V



Hình 2: Module còi

Module sử dụng transistor 9012 với điện áp hoạt động 3V3- 5VDC. Có 3 Chân GND, VCC, I/O tiện lợi cho việc giao tiếp bên ngoài, gắn vào các module điều khiển. Thường dùng cho các mạch báo động, chống trộm,... [2]

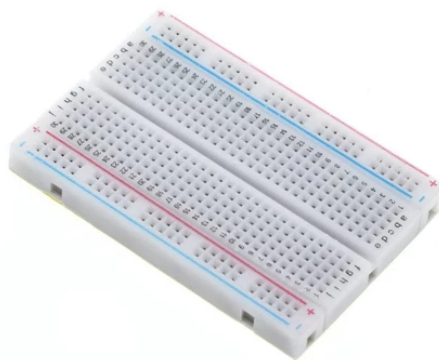
2.1.3 Module cảm biến PIR HC-SR501



Hình 3: Cảm biến PIR HC-SR501

Cảm biến thân nhiệt chuyển động PIR (Passive infrared sensor) HC-SR501 được sử dụng để phát hiện chuyển động của các vật thể phát ra bức xạ hồng ngoại (con người, con vật, các vật phát nhiệt,...), cảm biến có thể chỉnh được độ nhạy để giới hạn khoảng cách bắt xa gần cũng như cường độ bức xạ của vật thể mong muốn, ngoài ra cảm biến còn có thể điều chỉnh thời gian kích trễ (giữ tín hiệu bao lâu sau khi kích hoạt) qua biến trở tích hợp sẵn. [3]

2.1.4 Board mạch cắm mini 400 Lỗ



Hình 4: Breadboard mini

Breadboard cắm linh kiện 400 lỗ là sản phẩm không thể thiếu trong quá trình học tập điện tử. Thường được sử dụng để thực hành lắp ráp các bảng mạch mẫu và kiểm tra linh kiện điện tử giúp bạn dễ dàng thực hiện các mạch điện thực tế trước khi hàn trực tiếp linh kiện lên board mạch đồng.

2.1.5 Dây cắm truyền dữ liệu đực - cái và đực - đực



Hình 5: Dây cắm truyền dữ liệu

Dây cắm truyền dữ liệu đực – cái và đực – đực là phụ kiện không thể thiếu trong các dự án điện tử, đặc biệt khi làm việc với breadboard, vi điều khiển như Arduino hoặc các module cảm biến. Với thiết kế nhiều màu sắc giúp dễ dàng phân biệt tín hiệu, các dây này hỗ trợ kết nối linh hoạt giữa các linh kiện mà không cần hàn, tiết kiệm thời gian và công sức. Loại đực – đực thường dùng để nối giữa các điểm trên breadboard, trong khi đực – cái phù hợp để kết nối giữa module và chân cắm. Nhờ tính tiện lợi, dễ sử dụng và tái sử dụng nhiều lần, dây jumper trở thành công cụ cơ bản cho cả người mới học lẫn người làm kỹ thuật chuyên nghiệp. [5]

2.1.6 Servo motor



Hình 6: Servo motor

Servo SG90 là một loại động cơ servo nhỏ gọn, được sử dụng phổ biến trong các dự án điện tử và robot điều khiển tự động. Servo có khả năng quay chính xác theo góc mong muốn nhờ tín hiệu PWM, rất phù hợp để điều khiển cánh tay robot, xe tự hành hoặc hệ thống đóng mở tự động. [6]

2.2. Hệ điều hành Ubuntu

Hệ thống sử dụng hệ điều hành ubuntu 24.04 để thuận tiện cho việc nạp code vào ESP32 và chạy server mượt mà. Ubuntu có cơ chế phân quyền rõ ràng, ít bị virus và mã độc hơn so với nhiều hệ điều hành khác. Các bản cập nhật bảo mật cũng được cung cấp thường xuyên. Ngoài ra còn tiêu tốn ít tài nguyên hơn nhiều hệ điều hành khác nên có thể chạy ổn định trên các máy tính cấu hình yếu hoặc cũ. [7]

2.3. Các phần mềm hỗ trợ

2.3.1 MQTTx

MQTTx là công cụ MQTT client giao diện đồ họa (GUI) đa nền tảng cho phép kết nối tới MQTT broker để publish/subscribe message, hỗ trợ MQTT v3/v5, TLS/SSL và phục vụ kiểm thử – giám sát hệ thống IoT realtime.

Vai Trò & Ứng Dụng Trong Đồ Án :

- Giám sát luồng MQTT: Dùng MQTTX để subscribe camera và quan sát ngay payload do ESP32 publish - giúp kiểm tra backend đang nhận đúng topic
- Mô phỏng thiết bị: publish JSON giả lập (ESP32) để kích hoạt luồng backend
- Kiểm tra TLS / auth: dùng MQTTX để xác thực kết nối tới HiveMQ Cloud (port 8883)
- Hỗ trợ điều tra sự cố Khi backend không fetch được ảnh hoặc không gửi alert, MQTTx giúp tách vấn đề: xác nhận broker có nhận message, kiểm tra nội dung message, xác định xem lỗi nằm ở ESP32, backend hay mạng. [8]

2.3.2 Arduino IDE 2.x

Arduino IDE 2.x được sử dụng trong đồ án là công cụ viết và nạp code cho ESP32 Camera. Tích hợp các kho thư viện và board giúp hỗ trợ việc thực hiện đồ án dễ dàng hơn cũng như monitor ESP32 Camera có hoạt động ổn định sau khi nạp không. [9]

2.3.3 Python 3.11+

Python chính là thành phần điều phối trung tâm xử lý logic phần backend, đảm bảo tất cả dữ liệu từ ESP32-CAM được xử lý và phản hồi đúng chức năng.

Các thư viện chính cần phải cài :

- Flask
- Request
- Boto3
- Paho-mqtt

Python được sử dụng với mục đích chính dùng để xử lý các tính năng có trong đồ án phần backend và khởi động server bằng thư viện Flask trong file app.py. [10]

2.3.4 Telegram BOT

Telegram Bot là chương trình tự động tương tác với Telegram thông qua API của Telegram. Trong đồ án, bot này dùng để gửi cảnh báo từ hệ thống an ninh tới người dùng qua ứng dụng Telegram.

Bot được cấu hình bằng TELEGRAM_BOT_TOKEN và TELEGRAM_CHAT_ID trong file config.py. Telegram Bot là kênh cảnh báo chính của đồ án. Khi backend phát hiện có người trong ảnh, bot sẽ:

- Gửi thông báo văn bản có hiển thị màu và ký tự
- Gửi ảnh kèm caption
- Gửi trạng thái hoạt động của cam ESP32-CAM. [11]

2.3.5 Git Và GitHub

Git và Github được dùng để lưu trữ code, theo dõi thay đổi của mã nguồn hoặc tài liệu trong quá trình phát triển đồ án :

- Lưu lại lịch sử thay đổi của code
- Quay lại phiên bản cũ khi cần
- Làm việc nhóm dễ dàng
- Theo dõi ai sửa gì, khi nào
- Tránh mất code. [12]

2.3.6 AWS Rekognition

AWS Rekognition là dịch vụ phân tích hình ảnh và video dựa trên AI của Amazon Web Services, cung cấp khả năng nhận diện khuôn mặt, đối tượng, văn bản và nhiều tính năng thị giác máy tính khác theo mô hình API có trả phí.

Nguyên tắc hoạt động:

- Nhận ảnh đầu vào — Client (backend Flask) gửi ảnh JPEG/PNG dưới dạng byte hoặc base64 đến AWS qua HTTPS.
- Phát hiện khuôn mặt (Face Detection) — Rekognition xác định vị trí và đặc điểm khuôn mặt trong ảnh (bounding box, landmarks).
- Trích xuất đặc trưng (Feature Extraction) — Mô hình Deep Learning của AWS chuyển khuôn mặt thành vector đặc trưng (face embedding).
- So khớp với Face Collection — Vector này được so sánh với các khuôn mặt đã đăng ký trong Collection bằng thuật toán đo khoảng cách cosine similarity.
- Trả về kết quả — Trả về trạng thái Match / No Match kèm Confidence Score (0–100%). Ngưỡng mặc định trong đồ án là 80%. [13]

2.4. Các giao thức

2.4.1. MQTT (Pub/Sub)

Khái niệm: MQTT là giao thức truyền thông nhẹ (lightweight) dành cho các thiết bị IoT có tài nguyên hạn chế và mạng không ổn định. MQTT hoạt động theo mô hình publish/subscribe, trong đó các thiết bị không giao tiếp trực tiếp mà thông qua một broker trung gian. Publisher gửi dữ liệu theo topic, và broker sẽ phân phối dữ liệu đến các subscriber tương ứng. Giao thức này phù hợp với môi trường băng thông thấp, độ trễ cao, như các thiết bị ESP32-CAM kết nối qua WiFi.

Vai trò trong hệ thống: Trong đồ án, MQTT được sử dụng như giao thức truyền sự kiện (event transport) giữa ESP32-CAM và backend Python. ESP32-CAM đóng vai trò Publisher, phát tín hiệu motion lên broker ngay khi PIR phát hiện chuyển động. Backend Flask đóng vai trò Subscriber, liên tục lắng nghe và phản ứng với từng sự kiện nhận được.

Cấu hình MQTT trong hệ thống:

Tham số	Giá trị
Broker	HiveMQ Cloud
Port	8883 (MQTT over TLS)
Authentication	Username / Password
Topic publish (ESP32)	camera/<device_id>/motion
Topic subscribe (Backend)	camera/+ /motion

Port 8883 là cổng chuẩn cho MQTT over TLS, giúp mã hóa toàn bộ dữ liệu giữa ESP32 và broker. Cả ESP32 và backend xác thực bằng username/password được cấu hình trong config.py. Topic camera/+ /motion sử dụng wildcard + để backend có thể subscribe tất cả camera mà không cần khai báo từng topic riêng.

Cách hoạt động:

ESP32-CAM dùng thư viện PubSubClient để kết nối MQTT. Khi PIR phát hiện chuyển động (sau debounce 2 giây), ESP32 gửi JSON gồm device_id, ip, timestamp, motion=true lên topic camera/<device_id>/motion.

Backend (mqtt_client.py) dùng paho-mqtt để subscribe camera/+ /motion, bật TLS nếu dùng port 8883. Khi nhận message, backend sẽ:

- Parse JSON từ payload MQTT.
- Lọc duplicate dựa trên cặp device_id + timestamp để tránh xử lý trùng cùng một sự kiện.
- Chỉ tiếp tục khi motion = true.
- Trích xuất IP của ESP32 từ payload, gọi HTTP GET đến http://<ip>:80/capture để lấy ảnh.
- Chạy AI detection và gửi cảnh báo Telegram nếu phát hiện người.

Ưu điểm MQTT: MQTT nhẹ, tiết kiệm tài nguyên, phù hợp ESP32. Mô hình publish/subscribe giúp dễ mở rộng thêm camera mà không cần sửa backend. Giao thức hoạt động ổn định trên mạng yếu nhờ cơ chế keep-alive, và TLS trên port 8883 đảm bảo an toàn dữ liệu truyền tải.

2.4.2 HTTP ,HTTPS , NTP

❖ HTTP (HyperText Transfer Protocol)

Khái niệm: HTTP là giao thức truyền thông tầng ứng dụng hoạt động theo mô hình request/response. Client gửi request đến server, server xử lý và trả về response kèm dữ liệu. HTTP phù hợp cho việc truyền tải dữ liệu có kích thước lớn như ảnh JPEG — điều mà MQTT không được thiết kế để thực hiện hiệu quả.

Vai trò trong hệ thống: HTTP được sử dụng cho hai luồng giao tiếp riêng biệt trong hệ thống:

- Luồng thứ nhất là giao tiếp giữa Backend và ESP32-CAM. Sau khi nhận event MQTT, backend lấy địa chỉ IP của ESP32 từ payload và gọi HTTP GET đến ESP32 để tải ảnh JPEG về. Luồng này tách biệt rõ ràng trách nhiệm: MQTT chỉ đảm nhiệm báo hiệu sự kiện nhẹ, còn ảnh có dung lượng lớn được truyền riêng qua HTTP.
- Luồng thứ hai là giao tiếp giữa ESP32 và Backend khi ESP32 upload ảnh trực tiếp qua HTTP POST /upload mà không qua MQTT. Đây là cơ chế ưu tiên khi kết nối HTTP khả dụng.

Luồng hoạt động HTTP:

1. Sensor PIR phát hiện chuyển động, ESP32 gửi event MQTT đến broker.
2. Backend nhận event, trích xuất IP của ESP32 từ JSON payload.
3. Backend gọi HTTP GET http://<esp32_ip>:80/capture.
4. ESP32 chụp ảnh và trả về file JPEG trong response body.
5. Backend nhận ảnh, lưu trữ và chuyển sang tầng AI phân tích.
6. Backend nhận kết quả và gửi thông báo tin nhắn đến Telegram Bot

Việc tách biệt HTTP (truyền ảnh) và MQTT (truyền sự kiện) giúp tối ưu hóa hiệu suất: MQTT xử lý nhanh với payload nhỏ, còn HTTP chịu trách nhiệm truyền dữ liệu nhị phân dung lượng lớn mà không làm tắc nghẽn kênh sự kiện.

❖ HTTPS (HTTP Secure)

Khái niệm: HTTPS là phiên bản bảo mật của HTTP, sử dụng giao thức TLS để mã hóa toàn bộ dữ liệu trao đổi giữa client và server. HTTPS đảm bảo ba tính chất bảo mật cốt lõi: tính bí mật (dữ liệu được mã hóa, bên thứ ba không đọc được), tính toàn vẹn (dữ liệu không bị chỉnh sửa trong quá trình truyền) và tính xác thực (xác nhận đúng server đích thông qua chứng chỉ SSL).

Vai trò trong hệ thống: Backend Python sử dụng HTTPS cho tất cả giao tiếp với các dịch vụ bên ngoài Internet:

- Gọi Telegram Bot API qua `https://api.telegram.org/bot<token>/sendMessage` và `sendPhoto` để gửi cảnh báo và ảnh đến người dùng.
- Gọi AWS Rekognition API thông qua thư viện `boto3`, toàn bộ request đến AWS được thực hiện qua HTTPS theo tiêu chuẩn bảo mật của Amazon Web Services.

HTTPS là bắt buộc trong các trường hợp này vì dữ liệu truyền đi chứa thông tin nhạy cảm: Telegram bot token có thể bị lợi dụng để chiếm quyền bot nếu lộ ra ngoài, ảnh khuôn mặt là dữ liệu sinh trắc học cần được bảo vệ, và AWS credentials phải được truyền bảo mật để tránh truy cập trái phép vào tài khoản cloud.

❖ NTP (Network Time Protocol)

Khái niệm: NTP là giao thức mạng dùng để đồng bộ hóa thời gian của các thiết bị trong mạng với nguồn thời gian chuẩn. NTP hoạt động trên UDP port 123, kết nối đến các NTP server công cộng như `pool.ntp.org` để lấy thời gian UTC chính xác. Với các thiết bị nhúng như ESP32, NTP là phương pháp duy nhất để có được giờ thực tế chính xác vì ESP32 không tích hợp đồng hồ thời gian thực (RTC) phần cứng.

Vai trò trong hệ thống: NTP chỉ được sử dụng ở phía ESP32-CAM (cả cam1 và cam2). Backend Python không sử dụng NTP trực tiếp mà chỉ nhận và sử dụng timestamp do ESP32 tạo ra và đính kèm trong MQTT payload. ESP32 sử dụng NTP cho bốn mục đích cụ thể:

- Đồng bộ giờ thực tế ngay sau khi kết nối WiFi thành công, đảm bảo ESP32 có thời gian chính xác trước khi bắt đầu ghi nhận sự kiện.
- Tạo timestamp chính xác cho mỗi sự kiện chuyển động được phát hiện, timestamp này được đính kèm trong JSON payload gửi lên MQTT.

- Cho phép backend so sánh và sắp xếp thứ tự các sự kiện đến từ nhiều camera khác nhau dựa trên cùng một mốc thời gian chuẩn.
- Nếu không có NTP, mỗi ESP32 sẽ tự đếm thời gian từ lúc khởi động (millis), dẫn đến timestamp không đồng nhất giữa các thiết bị và làm sai lệch toàn bộ logic điều phối đa camera của hệ thống.

2.5. Đề xuất giải pháp

Đồ án Hệ thống an ninh thông minh sử dụng ESP32-CAM và AI nhận diện mang tính ứng dụng cao trong lĩnh vực giám sát và an ninh, với khả năng triển khai linh hoạt trong nhiều môi trường khác nhau với chi phí thấp.

Hệ thống được đề xuất triển khai với mô hình gồm 2 bộ ESP32-CAM, mỗi bộ tích hợp cảm biến PIR, buzzer cảnh báo và servo điều khiển. Hai camera hoạt động theo mô hình multi-camera, giúp tăng phạm vi giám sát và giảm điểm mù trong hệ thống.

Để mô phỏng thực tế, hệ thống được lắp đặt trên mô hình nhà thu nhỏ, trong đó các thiết bị được bố trí tại các vị trí như cửa ra vào và khu vực giám sát chính. Servo được sử dụng để mô phỏng cơ chế đóng/mở cửa tự động dựa trên kết quả nhận diện từ hệ thống AI.

Giải pháp này có các ưu điểm:

- Chi phí thấp, phù hợp với môi trường học tập
- Dễ dàng lắp đặt và triển khai thực nghiệm
- Linh kiện nhỏ gọn, dễ di chuyển và tái sử dụng
- Thể hiện đầy đủ kiến trúc hệ thống IoT kết hợp AI trong thực tế

PHẦN III. THIẾT KẾ VÀ PHÂN TÍCH HỆ THỐNG

3.1 Thiết kế hệ thống

3.1.1 Danh sách thiết bị và chi phí

Bảng linh kiện sử dụng:

	Combo 10 con - LED 5mm màu ánh sáng Đỏ, Xanh Lá, Xanh Dương, Vàng Phân loại hàng: 10 Led5mm-Xanh Lá x1	7.000đ
	Combo 10 con - LED 5mm màu ánh sáng Đỏ, Xanh Lá, Xanh Dương, Vàng Phân loại hàng: 10 Led 5mm - Đỏ x1	7.000đ
	Module Cảm Biến Chuyển Động PIR HC-SR501 (Cảm Biến Thân Nhiệt) Phân loại hàng: SR501 - Xanh dương x1	20.000đ
	Module Cảm Biến Chuyển Động PIR HC-SR501 (Cảm Biến Thân Nhiệt) Phân loại hàng: SR501 - Xanh lá x1	20.000đ
	Combo 65 hoặc 30 Dây Cắm Đực Đực - Dây cắm test board Phân loại hàng: Combo 65 sợi Đực-Đực x1	25.000đ
	Module Còi Chip 3.3V-5V, dùng để phát ra âm thanh khi kích tín hiệu Phân loại hàng: Module còi chip Low x2	28.000đ
	Board test cắm Mini 400 lỗ 8.5cm × 5.5cm - Bread board hàng tốt chất lượng cao Phân loại hàng: Board 400 x2	40.000đ 32.000đ
	Module thu phát wifi camera ESP32-CAM kèm Camera OV3660 tích hợp wifi, camera - chuyên dụng và bluetooth 4 Phân loại hàng: ESP32-CAM + Mạch Nạp x2	480.000đ
Tổng tiền hàng		619.000đ

Hình 7: Bảng chi phí các linh kiện sử dụng

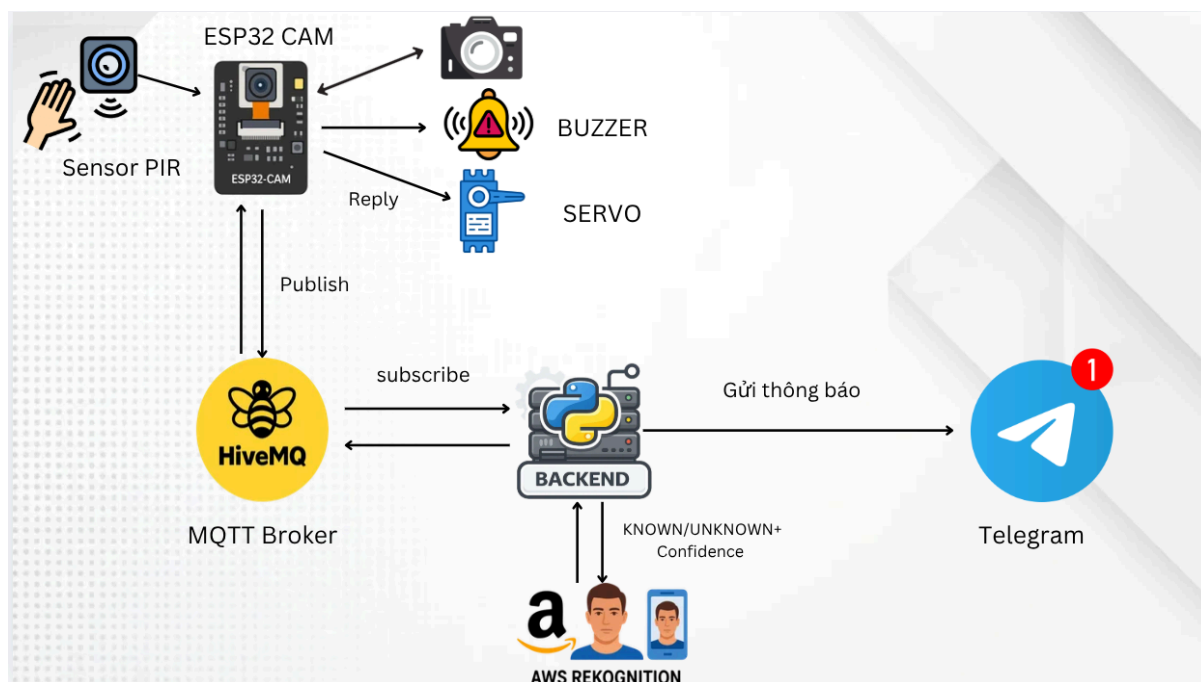
STT	Linh kiện	Số lượng
1	ESP32-CAM	2
2	PIR HC-SR501	2
3	Servo SG90	2
4	Buzzer	2
5	Breadboard	2
6	Dây Jumper	30

STT	Linh kiện	Số lượng
1	ESP32-CAM	2
7	Đèn led	20

Chi phí triển khai:

Thiết bị	Số lượng	Đơn giá	Thành tiền
ESP32-CAM	2	240.000	480.000
PIR	2	20.000	40.000
Servo	2	35.000	70.000
Buzzer	2	14.000	28.000
Breadboard	2	16.000	32.000
Bộ dây nối	1	25.000	25.000
Led mini	20	7.000	14.000
Tổng tiền	689.000 VNĐ		

3.1.2 Sơ đồ kiến trúc hệ thống



Hình 8: Sơ đồ kiến trúc hệ thống

Hệ thống Smart Security System được xây dựng trên nền tảng kiến trúc IoT phân tầng, tích hợp chặt chẽ giữa bốn lớp chính: lớp thiết bị nhúng (ESP32-CAM và các cảm biến ngoại vi), lớp truyền thông (MQTT over HiveMQ), lớp xử lý trung tâm (Backend Python – Flask) và lớp dịch vụ đám mây (AWS Rekognition và Telegram Bot).

Điểm cốt lõi của kiến trúc là mô hình event-driven (hướng sự kiện) - hệ thống không hoạt động liên tục mà chỉ kích hoạt khi có sự kiện thực sự xảy ra. Toàn bộ chuỗi xử lý được khởi động bởi một tín hiệu duy nhất từ cảm biến PIR, giúp tối ưu hóa tài nguyên phần cứng, giảm tiêu thụ điện năng và loại bỏ xử lý dư thừa.

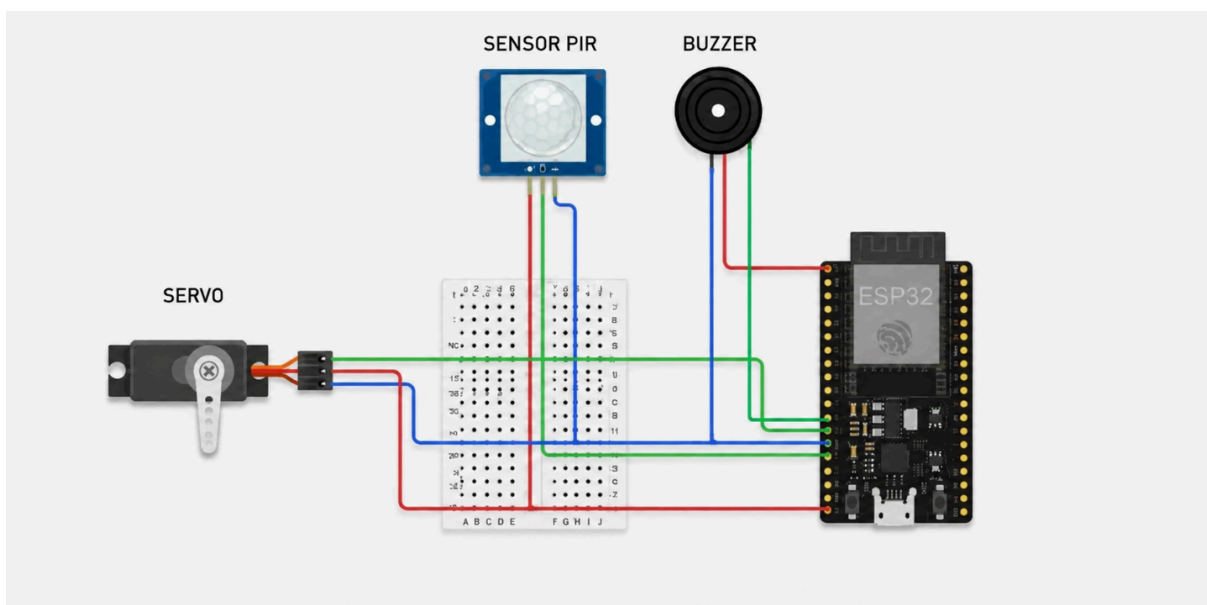
Hệ thống có các lớp như sau:

- Lớp 1 – Hardware (Thiết bị ngoại vi):** Cảm biến PIR đóng vai trò là bộ kích hoạt đầu tiên của toàn bộ hệ thống. Khi phát hiện chuyển động trong vùng giám sát, PIR gửi tín hiệu mức cao (HIGH) đến ESP32-CAM qua chân GPIO 13. ESP32-CAM lập tức tiến hành chụp ảnh JPEG độ phân giải 640x480 mp và chuẩn bị truyền dữ liệu. Buzzer được kết nối để phát cảnh báo âm thanh tức thì ngay tại thiết bị, trong khi Servo SG90 đảm nhiệm vai trò điều khiển cơ cấu khóa cửa vật lý dựa trên kết quả nhận diện.
- Lớp 2 – Communication (Truyền thông):** ESP32-CAM đóng vai trò MQTT Publisher, publish dữ liệu ảnh lên HiveMQ Broker thông qua topic camera/photo. HiveMQ là MQTT Broker hoạt động trên nền đám mây, đảm

bảo việc chuyển tiếp message đến Backend đang subscribe ổn định, có độ trễ thấp và hỗ trợ xác thực bảo mật. Chiều ngược lại, Backend cũng publish lệnh điều khiển qua MQTT để ESP32-CAM kích hoạt Servo và Buzzer theo kết quả xử lý.

- **Lớp 3 – Backend Processing (Xử lý trung tâm):** Flask Backend là trung tâm điều phối toàn bộ logic nghiệp vụ của hệ thống. Backend đảm nhiệm ba nhiệm vụ chính: nhận và giải mã dữ liệu ảnh từ MQTT, gọi AWS Rekognition API để phân tích khuôn mặt, và ra quyết định điều khiển dựa trên kết quả trả về. Backend hoạt động tại localhost:5000, đồng thời duy trì cơ chế cooldown 30 giây giữa các lần cảnh báo để tránh spam thông báo.
- **Lớp 4 – Cloud Services (Dịch vụ đám mây):** AWS Rekognition được tích hợp như một external API, nhận ảnh từ Backend qua HTTPS và trả về kết quả nhận diện gồm trạng thái KNOWN/UNKNOWN cùng Confidence Score với độ chính xác lên đến 99%. Telegram Bot API được sử dụng để đẩy thông báo tức thì đến người dùng cuối, với nội dung cảnh báo khác nhau tùy theo kết quả nhận diện.

3.1.3 Sơ đồ mạch



Hình 9: Sơ đồ mạch

Sơ đồ mạch của hệ thống bao gồm ESP32-CAM là vi điều khiển trung tâm, kết nối với ba thiết bị ngoại vi: cảm biến PIR, Buzzer và Servo SG90. Toàn bộ mạch được lắp ráp trên breadboard để tiện chỉnh sửa và kiểm tra trong quá trình phát triển.

❖ Bước 1 – Cấp nguồn cho hệ thống

Nguồn điện được phân phối từ ESP32-CAM ra toàn bộ breadboard:

- Chân 5V của ESP32 → nối vào hàng nguồn dương (+) của breadboard
- Chân GND của ESP32 → nối vào hàng nguồn âm (–) của breadboard

Việc đưa nguồn ra breadboard trước giúp các thiết bị ngoại vi dùng chung một nguồn cấp ổn định, tránh tình trạng sụt áp khi nhiều thiết bị hoạt động đồng thời.

❖ Bước 2 – Kết nối cảm biến PIR

Cảm biến PIR (HC-SR501) gồm 3 chân chức năng:

Chân PIR	Kết nối
VCC	Hàng nguồn dương (+) trên breadboard
GND	Hàng nguồn âm (–) trên breadboard
OUT	GPIO 13 của ESP32-CAM

Chân OUT của PIR xuất tín hiệu mức HIGH khi phát hiện chuyển động hồng ngoại trong vùng giám sát. GPIO 13 được lập trình ở chế độ INPUT để liên tục đọc trạng thái tín hiệu này.

❖ Bước 3 – Kết nối Buzzer

Buzzer chủ động (active buzzer) gồm 3 chân:

Chân Buzzer	Kết nối
Chân + (Anode)	Chân 3V3 của ESP32-CAM
Chân I/O (tín hiệu)	GPIO 14 của ESP32-Cam
Chân – (Cathode)	GND

Buzzer được kết nối trực tiếp vào chân 3V3 thay vì qua GPIO để đảm bảo âm lượng cảnh báo đủ lớn. Trong firmware, ESP32-CAM điều khiển buzzer thông qua một transistor hoặc relay nhỏ nếu cần tắt/bật chủ động.

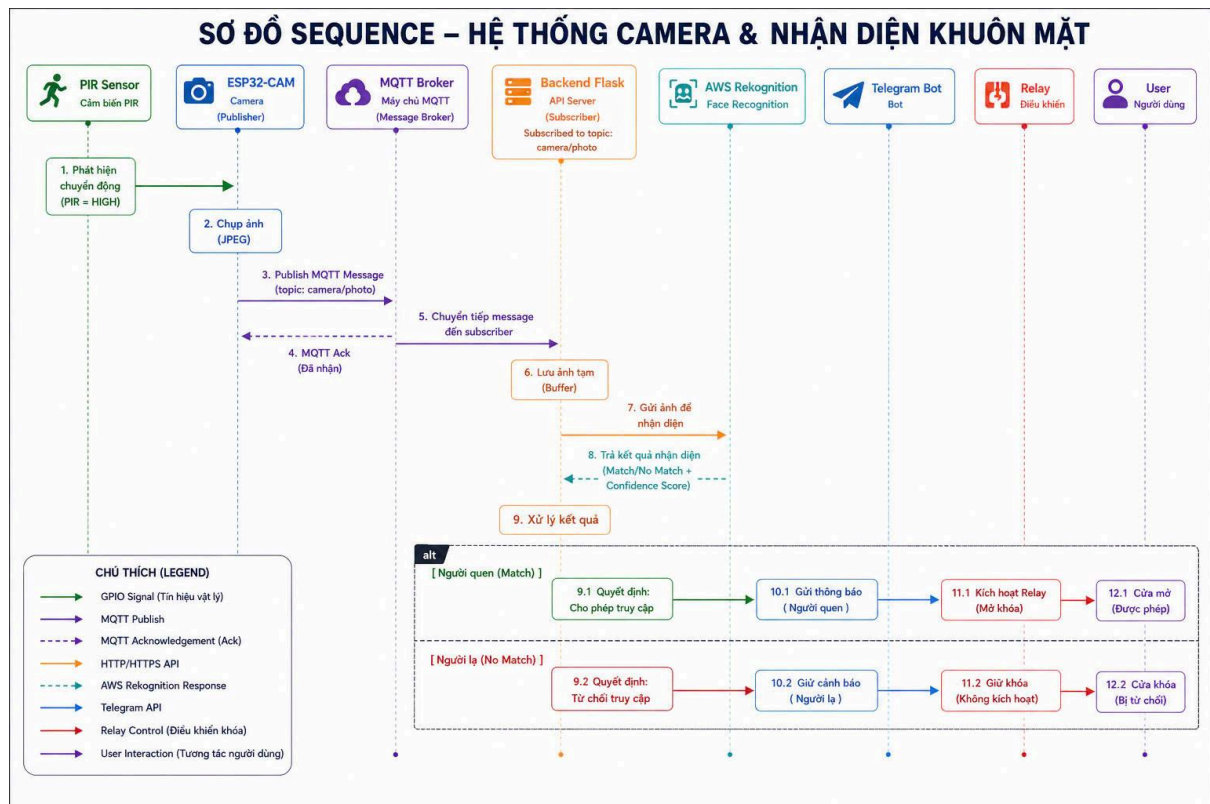
❖ Bước 4 – Kết nối Servo SG90

Servo SG90 gồm 3 dây màu với chức năng phân biệt rõ ràng:

Dây Servo	Màu	Kết nối
VCC	Đỏ	Hàng nguồn dương (+) trên breadboard
GND	Nâu hoặc Xanh đen	Hàng nguồn âm (–) trên breadboard
Signal	Cam hoặc Vàng	GPIO 12 của ESP32-CAM

Chân GPIO 12 xuất tín hiệu PWM để điều khiển góc quay của Servo. Khi hệ thống nhận diện đúng người quen, Backend gửi lệnh qua MQTT để ESP32-CAM phát xung PWM tương ứng góc mở (90°), và trả về 0° khi đóng lại.

3.1.4 Sơ đồ hoạt động



Hình 10: Sơ đồ luồng hoạt động

Hệ thống vận hành theo một chuỗi sự kiện tuần tự, được chia thành hai giai đoạn chính: giai đoạn thu thập và truyền dữ liệu (từ phần cứng lên Backend) và giai đoạn xử lý và phản hồi (từ Backend ra các thiết bị đầu ra và người dùng).

❖ Giai đoạn 1 – Thu thập và truyền dữ liệu

Bước 1: Cảm biến PIR liên tục quét vùng giám sát. Khi phát hiện bức xạ hồng ngoại từ người, chân OUT xuất tín hiệu HIGH và truyền đến GPIO 13 của ESP32-CAM. Cơ chế debounce 2 giây được áp dụng để lọc nhiễu tín hiệu, tránh kích hoạt nhầm.

Bước 2: ESP32-CAM nhận tín hiệu kích hoạt, khởi động module camera OV2640 và chụp ảnh.

Bước 3: ESP32-CAM thiết lập kết nối đến HiveMQ Broker qua giao thức MQTT (port 8883) với xác thực username/password, sau đó publish ảnh JPEG dưới dạng binary payload lên topic camera/photo.

Bước 4: HiveMQ Broker xác nhận đã nhận message thành công bằng MQTT Acknowledgement (ACK) gửi về ESP32-CAM, đảm bảo dữ liệu không bị mất trong quá trình truyền.

Bước 5: Flask Backend đang subscribe topic camera/photo nhận được message từ HiveMQ Broker. MQTT client chạy trong một thread riêng biệt, không ảnh hưởng đến luồng xử lý chính của Flask server.

Bước 6: Backend giải mã binary payload, tái tạo dữ liệu ảnh JPEG và lưu tạm vào bộ nhớ đệm (buffer) để chuẩn bị cho bước phân tích.

❖ Giai đoạn 2 – Nhận diện và phản hồi

Bước 7: Backend gọi AWS Rekognition API qua HTTPS, truyền ảnh JPEG dưới dạng base64 và yêu cầu so sánh với Face Collection đã được đăng ký trước trong hệ thống. Ngưỡng Confidence Score mặc định được đặt ở mức 80%.

Bước 8: AWS Rekognition phân tích ảnh và trả về kết quả gồm hai thông tin quan trọng: trạng thái nhận diện (Match / No Match) và Confidence Score (0–100%) cho biết mức độ tin cậy của kết quả. Nếu không phát hiện khuôn mặt nào trong ảnh, AWS trả về trạng thái No Face Detected.

Bước 9: Backend nhận kết quả và thực hiện phân nhánh xử lý theo hai trường hợp:

❖ Trường hợp 1 – Người quen (Match)

Đây là trường hợp AWS Rekognition trả về kết quả khớp với một khuôn mặt đã đăng ký trong hệ thống với Confidence Score $\geq 80\%$.

Bước 9.1: Backend xác nhận danh tính hợp lệ, ghi nhận thời gian truy cập và đưa ra quyết định cho phép truy cập.

Bước 10.1: Backend gọi Telegram Bot API để gửi thông báo đến người dùng, nội dung bao gồm tên người được nhận diện, Confidence Score, thời gian và ảnh chụp tại thời điểm phát hiện.

Bước 11.1: Backend publish lệnh điều khiển qua MQTT về ESP32-CAM. ESP32-CAM nhận lệnh và phát xung PWM đến GPIO 12, kích hoạt Servo SG90 quay về góc 90° để mở khóa cửa.

Bước 12.1: Cửa mở ra, người dùng được phép vào. Sau khoảng thời gian cài đặt (mặc định 5 giây), Servo tự động quay về vị trí 0° để đóng khóa lại.

❖ Trường hợp 2 – Người lạ (No Match)

Đây là trường hợp AWS Rekognition không tìm thấy khuôn mặt khớp trong collection, hoặc Confidence Score dưới ngưỡng cho phép, hoặc không phát hiện khuôn mặt nào trong ảnh.

Bước 9.2: Backend xác nhận danh tính không hợp lệ, ghi nhận sự kiện xâm nhập và đưa ra quyết định từ chối truy cập.

Bước 10.2: Backend gọi Telegram Bot API để gửi cảnh báo khẩn cấp đến người dùng, nội dung bao gồm thông báo phát hiện người lạ, thời gian, camera phát hiện và ảnh chụp kèm theo.

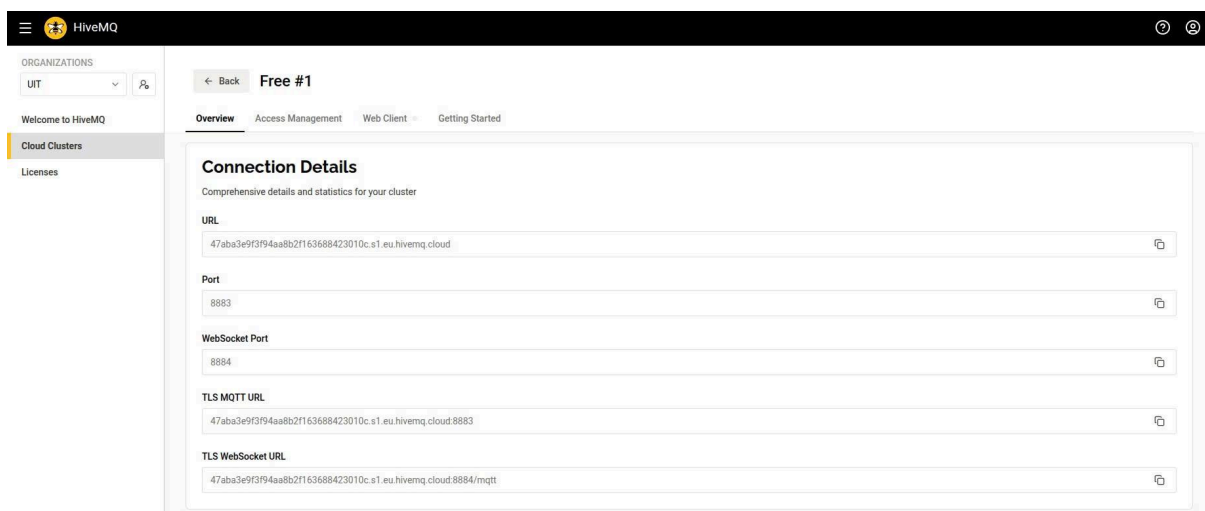
Bước 11.2: Backend publish lệnh điều khiển Buzzer qua MQTT về ESP32-CAM. Relay/Servo không được kích hoạt, cửa giữ nguyên trạng thái khóa.

Bước 12.2: Cửa vẫn khóa. Ảnh của người lạ được lưu tự động vào thư mục unknown_faces/ trên server với tên file gắn timestamp để phục vụ tra cứu sau này. Người dùng có thể xem lại cảnh báo và ảnh trực tiếp trên Telegram.

3.1.5 Cấu hình MQTT Broker (HiveMQ Cloud)

Trong hệ thống Smart Security System, MQTT được sử dụng làm giao thức truyền thông chính giữa các thiết bị ESP32-CAM và Backend Flask. Để đảm bảo khả năng hoạt động ổn định, hỗ trợ nhiều thiết bị kết nối đồng thời và dễ dàng triển khai từ xa, nhóm lựa chọn HiveMQ Cloud làm MQTT Broker trung tâm.

HiveMQ Cloud hoạt động theo mô hình Publish/Subscribe, trong đó ESP32-CAM đóng vai trò Publisher gửi các sự kiện phát hiện chuyển động lên Broker, còn Backend Flask đóng vai trò Subscriber nhận dữ liệu và thực hiện các tác vụ xử lý tiếp theo. Việc sử dụng Broker trung gian giúp các thiết bị không cần kết nối trực tiếp với nhau, từ đó tăng khả năng mở rộng và giảm độ phức tạp của hệ thống.



Hình 11: Thông số kết nối HiveMQ Cloud cluster

Hình 11 thể hiện thông tin kết nối chi tiết của HiveMQ Cloud cluster được sử dụng trong hệ thống, bao gồm URL broker, cổng MQTT tiêu chuẩn (8883) và cổng WebSocket (8884).

Hệ thống sử dụng kết nối MQTT over TLS trên cổng 8883 nhằm đảm bảo tính bảo mật của dữ liệu trong quá trình truyền tải. Tất cả các thiết bị đều phải xác thực bằng Username và Password trước khi được phép kết nối đến Broker. Bảng sau trình bày các thông số cấu hình MQTT được sử dụng trong hệ thống.

Tham số	Giá trị
MQTT Broker	HiveMQ Cloud
Giao thức	MQTT v3.1.1
Port	8883
Bảo mật	TLS/SSL
Authentication	Username / Password
Publish Topic	camera/<device_id>/motion
Subscribe Topic	camera/+ /motion
Keep Alive	60 giây

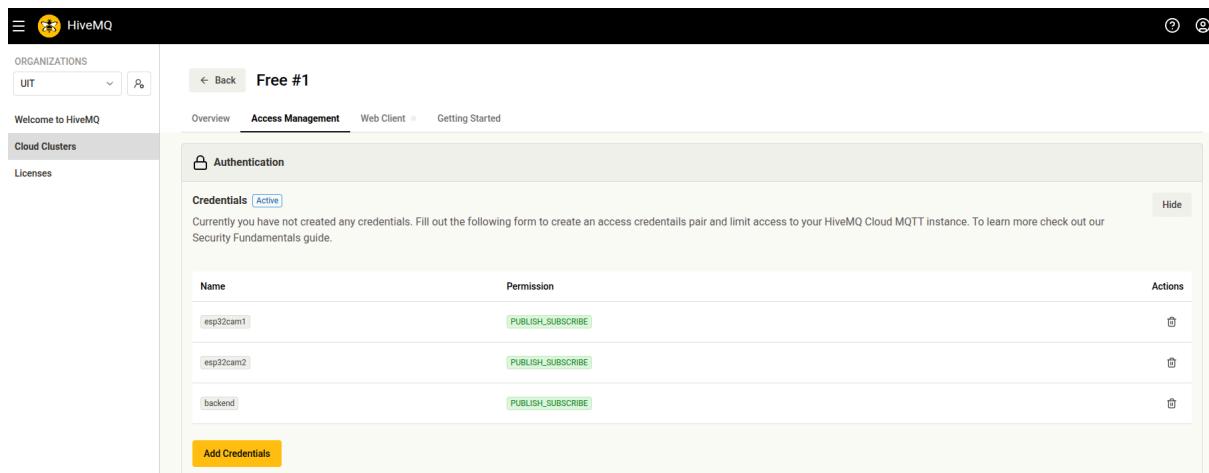
Cấu hình MQTT được khai báo trong file config.py của backend. Các thông số bao gồm địa chỉ broker HiveMQ Cloud, cổng 8883 (MQTT over TLS),

thông tin xác thực và topic filter camera/+/motion với keepalive 60 giây.

```
28 # MQTT broker configuration
29 MQTT_BROKER_HOST = '47aba3e9f3f94aa8b2f163688423010c.s1.eu.hivemq.cloud'
30 MQTT_BROKER_PORT = 8883
31 MQTT_USERNAME = 'backend'
32 MQTT_PASSWORD = 'Nhutkhanh2025'
33 MQTT_TOPIC_FILTER = 'camera/+/motion'
34 MQTT_KEEPALIVE = 60
35 MQTT_DEBUG = False
36
```

Hình 12: Cấu hình MQTT broker trong config.py

Để kiểm soát quyền truy cập vào broker, nhóm tạo ba bộ credentials riêng biệt tương ứng với ba thực thể trong hệ thống: esp32cam1, esp32cam2 và backend. Tất cả đều được cấp quyền PUBLISH_SUBSCRIBE, cho phép vừa gửi vừa nhận dữ liệu thông qua broker. Việc phân tách credentials theo từng thiết bị giúp hệ thống dễ dàng thu hồi quyền truy cập của một thiết bị cụ thể mà không ảnh hưởng đến các thành phần còn lại.



Hình 13: Danh sách credentials được cấu hình trên HiveMQ Cloud

Trong đó, mỗi ESP32-CAM được gán một định danh riêng (device_id) như cam1 hoặc cam2. Khi cảm biến PIR phát hiện chuyển động, ESP32-CAM sẽ gửi một thông điệp JSON đến topic tương ứng theo định dạng sau:

```
{
  "device_id": "cam1",
  "motion": true,
  "ip": "192.168.1.100",
  "timestamp": "2026-05-20T14:30:25"
}
```

Hình 14: Thông điệp json ESP32-CAM gửi đến topic tương ứng

Backend Flask subscribe topic camera/+/motion bằng cơ chế wildcard (+), cho phép nhận dữ liệu từ tất cả các camera mà không cần cấu hình riêng cho từng thiết bị. Sau khi nhận được thông điệp MQTT, Backend sẽ tiến hành lấy ảnh từ ESP32-CAM thông qua HTTP, thực hiện nhận diện khuôn mặt bằng AWS Rekognition và gửi cảnh báo đến Telegram nếu phát hiện sự kiện bất thường.

Việc sử dụng HiveMQ Cloud kết hợp MQTT giúp hệ thống đạt được các ưu điểm như giảm băng thông truyền tải, hỗ trợ giao tiếp thời gian thực, dễ dàng mở rộng số lượng camera và đảm bảo tính ổn định khi triển khai trong môi trường Internet thực tế.

3.2 Phân tích chức năng và khởi động hệ thống

3.2.1 Phân tích chức năng

Hệ thống được tổ chức theo mô hình phân tầng với bảy thành phần phần mềm độc lập, mỗi thành phần đảm nhiệm một vai trò cụ thể trong toàn bộ luồng xử lý từ phần cứng đến cảnh báo người dùng.

```

project/
├── esp32/
│   ├── cam1/
│   │   └── main.ino          # ESP32 MQTT publisher for Camera 1
│   ├── cam2/
│   │   └── main.ino          # ESP32 MQTT publisher for Camera 2
│   └── backend/
│       ├── app.py            # Flask server + MQTT subscriber launcher
│       ├── mqtt_client.py     # MQTT client subscribing to camera events
│       ├── config.py          # Configuration settings
│       ├── ai/
│       │   ├── detect.py      # AWS Face detection & recognition
│       │   ├── telegram_alerts.py # Telegram alert integration
│       │   ├── example_usage.py # Usage examples
│       │   ├── quick_start.py  # Quick start patterns
│       │   └── config_helper.py # Configuration helper
│       ├── coordination/
│       │   └── coordinator.py  # Multi-camera coordination
│       ├── services/
│       │   └── telegram.py     # Telegram notification service
│       ├── known_faces.json    # Face ID ↔ Name mapping (auto-created)
│       ├── unknown_faces/     # Unknown faces storage (auto-created)
│       ├── requirements.txt     # Python dependencies
│       └── README.md           # This file

```

Hình 15: Cấu trúc file hệ thống

a) Firmware ESP32-CAM – esp32/cam1/main.ino, esp32/cam2/main.ino

Firmware được nạp riêng biệt cho từng module ESP32-CAM, tương ứng với định danh cam1 và cam2. Việc tách riêng file firmware cho phép backend phân biệt nguồn gốc dữ liệu khi hệ thống vận hành đa camera.

Các chức năng chính của firmware:

- Kết nối WiFi và duy trì kết nối MQTT đến broker tại port 8883 với hỗ trợ mã hóa TLS.
- Khởi chạy HTTP server nội bộ trên cổng 80 phục vụ ba endpoint:
 - GET /capture – chụp và trả về ảnh JPEG theo yêu cầu từ backend
 - GET /health – trả về trạng thái hoạt động của thiết bị
 - GET /test – chụp ảnh thử để kiểm tra kết nối camera
- Đọc tín hiệu từ cảm biến PIR tại chân GPIO 13. Firmware áp dụng cơ chế debounce 2 giây để lọc nhiễu và cooldown 15 giây giữa các lần xử lý liên tiếp, đảm bảo mỗi sự kiện chuyển động chỉ được xử lý đúng một lần.
- Khi chuyển động được xác nhận ổn định, ESP32-CAM kích hoạt flash LED tại GPIO 4, tiến hành chụp ảnh JPEG và ưu tiên upload trực tiếp lên backend qua HTTP POST /upload. Trong trường hợp upload thất bại, ESP32-CAM tự động chuyển sang publish event MQTT kèm địa chỉ IP của thiết bị để backend chủ động kéo ảnh về.

- Điều khiển còi buzzer tại GPIO 14 để phát cảnh báo âm thanh trực tiếp tại thiết bị khi phát hiện chuyển động.

b) Backend Flask – app.py

app.py là điểm khởi động trung tâm của backend, đồng thời khởi tạo và liên kết tất cả các thành phần con. Khi được chạy, app.py thực hiện ba tác vụ song song: khởi tạo Flask server, tạo instance MultiCameraCoordinator để quản lý logic cảnh báo đa camera, và khởi động MQTT client thông qua init_mqtt(coordinator) trên một thread riêng biệt.

Backend cung cấp bốn endpoint HTTP:

Endpoint	Phương thức	Chức năng
/upload	POST	Nhận ảnh multipart từ ESP32, gọi AI detection, xử lý và gửi cảnh báo
/status	GET	Trả về trạng thái backend, kết nối MQTT, danh sách ESP32 đã kết nối
/health	GET	Kiểm tra sức khỏe server
/	GET	Thông tin hệ thống và danh sách endpoint khả dụng

Endpoint /upload là luồng xử lý chính: nhận ảnh từ ESP32, lưu ảnh vào thư mục captured_images nếu SAVE_IMAGES = True, gọi hàm detect_human từ module AI, và chuyển kết quả sang coordinator để quyết định có phát sinh cảnh báo hay không.

c) MQTT Client – mqtt_client.py

MqttBackendClient xử lý toàn bộ luồng nhận dữ liệu từ kênh MQTT, hoạt động độc lập song song với Flask server. Module subscribe topic camera/+/motion để nhận event từ tất cả camera theo cơ chế wildcard.

Luồng xử lý khi nhận MQTT message diễn ra tuần tự như sau:

1. Giải mã JSON payload từ message nhận được.
2. Lọc duplicate dựa trên cặp device_id + timestamp để tránh xử lý trùng cùng một sự kiện.
3. Chỉ tiếp tục xử lý khi trường motion = true trong payload.
4. Lưu device_id vào danh sách connected_esp32_devices để endpoint /status theo dõi.

- Trích xuất địa chỉ IP của ESP32 từ payload, gọi HTTP đến `http://<ip>:80/capture` để tải ảnh về, có cơ chế retry khi thất bại.
- Lưu ảnh cục bộ nếu `SAVE_IMAGES = True`, sau đó gửi ảnh qua `detect_human` để phân tích.
- Nếu phát hiện người, gọi `coordinator.add_detection(...)` để nhận quyết định từ coordinator, và gọi `send_alert(...)` nếu coordinator cho phép gửi cảnh báo.

Module hỗ trợ kết nối TLS đầy đủ khi broker hoạt động trên port 8883.

d) AI Detection – ai/detect.py

HumanDetector là module phân tích trí tuệ nhân tạo, sử dụng phương pháp được cấu hình qua `AI_METHOD` trong `config.py`. Module khởi tạo AWS Rekognition client với thông tin xác thực `AWS_REGION`, `AWS_ACCESS_KEY`, `AWS_SECRET_KEY` và thực hiện hai tầng phân tích:

- Gọi `detect_labels` để phát hiện sự hiện diện của con người trong ảnh thông qua các nhãn `Person`, `Human`, `People`. Kết quả được lọc theo `CONFIDENCE_THRESHOLD` để loại bỏ các phát hiện có độ tin cậy thấp.
- So sánh khuôn mặt trong ảnh với database khuôn mặt đã đăng ký để phân biệt người quen (`KNOWN`) và người lạ (`UNKNOWN`), trả về kết quả kèm `Confidence Score`.

```

12 # AI Detection configuration
13 AI_METHOD = 'aws'
14 CONFIDENCE_THRESHOLD = 0.5 # Minimum confidence for detection
15
16 # AWS Rekognition (using AWS)
17 AWS_REGION = 'us-east-1'
18 AWS_ACCESS_KEY = 'AKIAXCRJJAXMZOW6U7MF' # Replace with real key
19 AWS_SECRET_KEY = 'oLv71+WoAcNv/OXW4pNScaL2caxrA22wo/EWkSBl' # Replace with real secret
20
21 # AWS Rekognition Face Recognition settings
22 AWS_FACE_COLLECTION_ID = 'smart-security-collection' # Rekognition collection for known faces
23 AWS_FACE_MATCH_THRESHOLD = 80.0 # Confidence threshold for face matching (0-100)
24 AWS_FACE_SIMILARITY_THRESHOLD = 80.0 # Similarity threshold for search_faces_by_image (0-100)
25 AWS_KNOWN_FACES_DB = 'known_faces.json' # Local JSON file for faceId <-> name mapping
26 AWS_UNKNOWN_FACES_DIR = 'unknown_faces/' # Directory to save unknown faces for review
27

```

Hình 16: Cấu hình AI Detection và AWS Rekognition trong `config.py`

Hình 16 thể hiện cấu hình AI và AWS Rekognition trong file `config.py`. Hệ thống sử dụng `AI_METHOD = 'aws'` với ngưỡng phát hiện `CONFIDENCE_THRESHOLD = 0.5`. Các thông số AWS bao gồm region `us-east-1`, access key, secret key và Face Collection ID là `smart-security-collection`. Ngưỡng so khớp khuôn mặt `AWS_FACE_MATCH_THRESHOLD` và `AWS_FACE_SIMILARITY_THRESHOLD` đều được đặt ở mức 80.0, đảm bảo cân bằng giữa độ chính xác và khả năng phát hiện. Khuôn mặt đã biết được lưu trong

known_faces.json, khuôn mặt lạ được lưu vào thư mục unknown_faces/ để phục vụ tra cứu sau này.

```
1  {
2    "320eebc4-9389-46a0-bed6-019e7d8abb7c": {
3      "name": "Khanh",
4      "registered_at": "2026-04-20T13:59:54.845254",
5      "confidence": 99.99995422363281
6    },
7    "dc7da3d7-1f30-42d9-9368-e0c2f32efed5": {
8      "name": "LamKhanh",
9      "registered_at": "2026-04-22T11:12:04.927578",
10     "confidence": 99.9988784790039
11   },
12   "9fe3e0ca-d06a-48e6-93f3-ae7b9adac593": {
13     "name": "Phat",
14     "registered_at": "2026-04-22T11:12:17.519571",
15     "confidence": 99.99959564208984
16   }
17 }
```

Hình 17: Nội dung file known_faces.json

Hình 17 thể hiện nội dung file known_faces.json — cơ sở dữ liệu ánh xạ giữa Face ID do AWS Rekognition cấp và thông tin người dùng tương ứng. Mỗi bản ghi bao gồm face_id (UUID do AWS tạo), tên người dùng (name), thời điểm đăng ký (registered_at) và confidence score tại thời điểm đăng ký. Trong quá trình thử nghiệm, hệ thống đã đăng ký ba khuôn mặt: Khanh, LamKhanh và Phat, với confidence score đạt trên 99.99% — cho thấy chất lượng ảnh đăng ký tốt và AWS Rekognition nhận diện chính xác.

e) Multi-Camera Coordinator – coordination/coordinator.py

MultiCameraCoordinator giải quyết bài toán quản lý cảnh báo khi nhiều camera hoạt động đồng thời, tránh tình trạng spam thông báo và phân loại mức độ sự kiện. Module duy trì ba cấu trúc dữ liệu nội bộ.

Coordinator kiểm tra ALERT_COOLDOWN_SECONDS trước khi trả về event cảnh báo. Nếu còn trong thời gian cooldown, hàm trả về None và không kích hoạt thông báo, giúp loại bỏ hiệu quả tình trạng spam Telegram khi cùng một sự kiện kéo dài.

f) Telegram Service – services/telegram.py

TelegramService đóng gói toàn bộ logic giao tiếp với Telegram Bot API. Module nhận cấu hình từ TELEGRAM_BOT_TOKEN và TELEGRAM_CHAT_ID,

xử lý hai trường hợp gửi thông báo thông qua hàm `send_alert(device_id, image_bytes, message)`:

- Nếu có dữ liệu ảnh: gửi photo kèm caption mô tả chi tiết sự kiện bao gồm camera nguồn, thời gian phát hiện và loại cảnh báo.
- Nếu không có ảnh: gửi tin nhắn text để đảm bảo người dùng vẫn nhận được thông báo ngay cả khi việc tải ảnh từ ESP32 thất bại.

g) Cấu hình hệ thống – `config.py`

`config.py` tập trung toàn bộ tham số cấu hình của hệ thống vào một nơi duy nhất, giúp việc triển khai và tùy chỉnh trở nên đơn giản mà không cần chỉnh sửa từng file riêng lẻ.

Nhóm cấu hình	Các tham số
Flask	Host, port, chế độ debug
Telegram	Bot token, chat ID
AI	Phương pháp nhận diện, confidence threshold
AWS	Region, access key, secret key, face collection ID
MQTT	Broker host, port, topic filter, keepalive, debug mode
Điều phối	Detection window (giây), alert cooldown (giây)
Lưu ảnh	Đường dẫn thư mục, bật/tắt tính năng

3.2.2 Khởi động hệ thống

Hệ thống backend được triển khai trên máy tính cá nhân hoặc máy chủ cục bộ. Yêu cầu tối thiểu là Python phiên bản 3.11 trở lên đã được cài đặt trên máy.

Yêu cầu thư viện:

Thư viện	Vai trò
Flask	Web framework cho backend server
Requests	Gọi HTTP đến ESP32 để lấy ảnh
Boto3	AWS SDK để tích hợp AWS Rekognition

paho-mqtt	MQTT client để kết nối và subscribe HiveMQ Broker
-----------	---

Bước 1 – Cài đặt dependencies:

```
cd backend
pip install -r ../requirements.txt
```

Hình 18: Lệnh cài đặt dependencies

Bước 2 – Kích hoạt môi trường ảo:

```
cd backend
# Activate virtual environment
source ../venv/bin/activate # On Windows: ../venv/Scripts/activate
python app.py
```

Hình 19: Lệnh kích hoạt môi trường ảo

Sau khi kích hoạt thành công, tên môi trường ảo xuất hiện ở đầu dòng lệnh xác nhận môi trường đã sẵn sàng.

Bước 3 – Cấu hình hệ thống:

Mở file config.py và điền đầy đủ các thông tin: MQTT broker host, Telegram bot token, Telegram chat ID, AWS credentials và các ngưỡng AI. Bước này bắt buộc phải hoàn thành trước khi khởi động server.

Bước 4 – Khởi động server:

Khi khởi động thành công với lệnh python app.py, terminal hiển thị xác nhận Flask server đang lắng nghe tại http://localhost:5000, MQTT client đã kết nối đến broker và bắt đầu subscribe topic camera/+/motion. Hệ thống sẵn sàng nhận dữ liệu từ các ESP32-CAM và xử lý cảnh báo ngay khi có chuyển động được phát hiện.

```

UnicodeEncodeError: 'charmap' codec can't encode character '\U0001f4e1' in position 47: character maps to <undefined>
Call stack:
  File "C:\Users\trung\AppData\Local\Programs\Python\Python311\Lib\threading.py", line 995, in _bootstrap
    self._bootstrap_inner()
  File "C:\Users\trung\AppData\Local\Programs\Python\Python311\Lib\threading.py", line 1038, in _bootstrap_inner
    self.run()
  File "C:\Users\trung\AppData\Local\Programs\Python\Python311\Lib\threading.py", line 975, in run
    self._target(*self._args, **self._kwargs)
  File "C:\Users\trung\AppData\Local\Programs\Python\Python311\Lib\site-packages\paho\mqtt\client.py", line 4523, in _thread_main
    self.loop_forever(retry_first_connection=True)
  File "C:\Users\trung\AppData\Local\Programs\Python\Python311\Lib\site-packages\paho\mqtt\client.py", line 2297, in loop_forever
    rc = self._loop(timeout)
  File "C:\Users\trung\AppData\Local\Programs\Python\Python311\Lib\site-packages\paho\mqtt\client.py", line 1686, in _loop
    rc = self._loop_read()
  File "C:\Users\trung\AppData\Local\Programs\Python\Python311\Lib\site-packages\paho\mqtt\client.py", line 2100, in _loop_read
    rc = self._packet_read()
  File "C:\Users\trung\AppData\Local\Programs\Python\Python311\Lib\site-packages\paho\mqtt\client.py", line 3142, in _packet_read
    rc = self._packet_handle()
  File "C:\Users\trung\AppData\Local\Programs\Python\Python311\Lib\site-packages\paho\mqtt\client.py", line 3814, in _packet_handle
    return self._handle_connack()
  File "C:\Users\trung\AppData\Local\Programs\Python\Python311\Lib\site-packages\paho\mqtt\client.py", line 3922, in _handle_connack
    on_connect()
  File "D:\pj\chung\Smart-Security-System-with-ESP32-CAM-and-AI-Detection\backend\mqtt_client.py", line 47, in _on_connect
    logger.info(f"MQTT subscribed to topic pattern: {MQTT_TOPIC_FILTER}")
Message: 'MQTT subscribed to topic pattern: camera/+motion'
Arguments: ()
2026-05-25 03:52:39,548 - mqtt_client - INFO - MQTT subscribed to topic pattern: camera/+motion
2026-05-25 03:52:39,550 - mqtt_client - INFO - Waiting for motion events from ESP32 cameras...

```

Hình 20: Backend Flask nhận và xử lý dữ liệu từ ESP32-CAM

PHẦN IV. XÂY DỰNG THỬ NGHIỆM VÀ ĐÁNH GIÁ

4.1 Môi trường thử nghiệm thực tế

Hệ thống được thử nghiệm trong điều kiện thực tế với môi trường lắp đặt mô phỏng các kịch bản giám sát thực tế, bao gồm cửa ra vào, hành lang và nhà mô hình demo thu nhỏ.

Yêu cầu môi trường lắp đặt:

- **Vị trí ESP32-CAM:** Đặt tại cửa ra vào, hành lang hoặc khu vực cần giám sát. Góc đặt camera cần đảm bảo vùng quan sát bao phủ điểm chuyển động chính, đồng thời giữ khoảng cách phù hợp để camera nhận diện khuôn mặt rõ ràng.
- **Vị trí PIR Sensor:** Hướng cảm biến về khu vực chuyển động chính. Tránh đặt PIR gần nguồn nhiệt, điều hòa hoặc cửa sổ có ánh sáng mặt trời trực tiếp để giảm thiểu cảnh báo giả do nhiễu nhiệt.
- **Mạng WiFi:** ESP32-CAM và máy chạy backend phải kết nối cùng mạng LAN, hoặc sử dụng MQTT broker có thể truy cập từ cả hai đầu nếu triển khai từ xa.

- **Nguồn điện:** Sử dụng nguồn 5V ổn định cho ESP32-CAM. Cần tránh sụt áp trong thời điểm camera chụp ảnh và flash LED hoạt động đồng thời, vì đây là thời điểm tiêu thụ dòng cao nhất.

Mục tiêu thử nghiệm:

Thử nghiệm được thiết kế để xác minh toàn diện từng thành phần và luồng xử lý đầu cuối của hệ thống, bao gồm:

- Xác nhận ESP32-CAM kết nối WiFi ổn định và gửi dữ liệu MQTT thành công đến broker.
- Kiểm tra PIR phát hiện chuyển động đúng thời điểm và kích hoạt chụp ảnh chính xác.
- Kiểm tra còi buzzer phát cảnh báo âm thanh khi phát hiện người và tự ngắt sau thời gian cài đặt.
- Đảm bảo backend nhận ảnh từ ESP32 và lưu đầy đủ vào thư mục `captured_images`.
- Kiểm tra AI nhận diện người hoạt động chính xác với cả hai phương pháp `AI_METHOD = 'aws'`, có khả năng phân biệt người quen và người lạ.
- Đảm bảo Telegram nhận được cảnh báo kèm ảnh đúng thời điểm khi có sự kiện kích hoạt.
- Thử nghiệm với nhiều ESP32-CAM hoạt động đồng thời để xác nhận logic phối hợp đa camera và khả năng giảm báo động giả của coordinator.
- Đánh giá cơ chế đóng mở cửa tự động thông qua relay hoạt động đúng theo kết quả nhận diện.

4.2 Kết quả thử nghiệm và đánh giá

4.2.1 Kết quả vận hành tổng thể

Hệ thống vận hành ổn định trong suốt quá trình thử nghiệm. ESP32-CAM duy trì kết nối WiFi và gửi dữ liệu MQTT liên tục trong các phiên thử nghiệm kéo dài 90 phút mà không phát sinh lỗi kết nối. Mỗi sự kiện PIR đều được xử lý đúng luồng: ESP32 gửi ảnh, backend nhận và lưu trữ vào `backend/captured_images`, sau đó kích hoạt chuỗi phân tích AI và gửi cảnh báo. Buzzer hoạt động chính xác khi có chuyển động và tự ngắt sau thời gian cài đặt. Toàn bộ giao tiếp mạng và MQTT không phát sinh lỗi lớn khi cấu hình broker và các thông số kết nối được thiết lập đúng.

4.2.2 Độ chính xác AI nhận diện

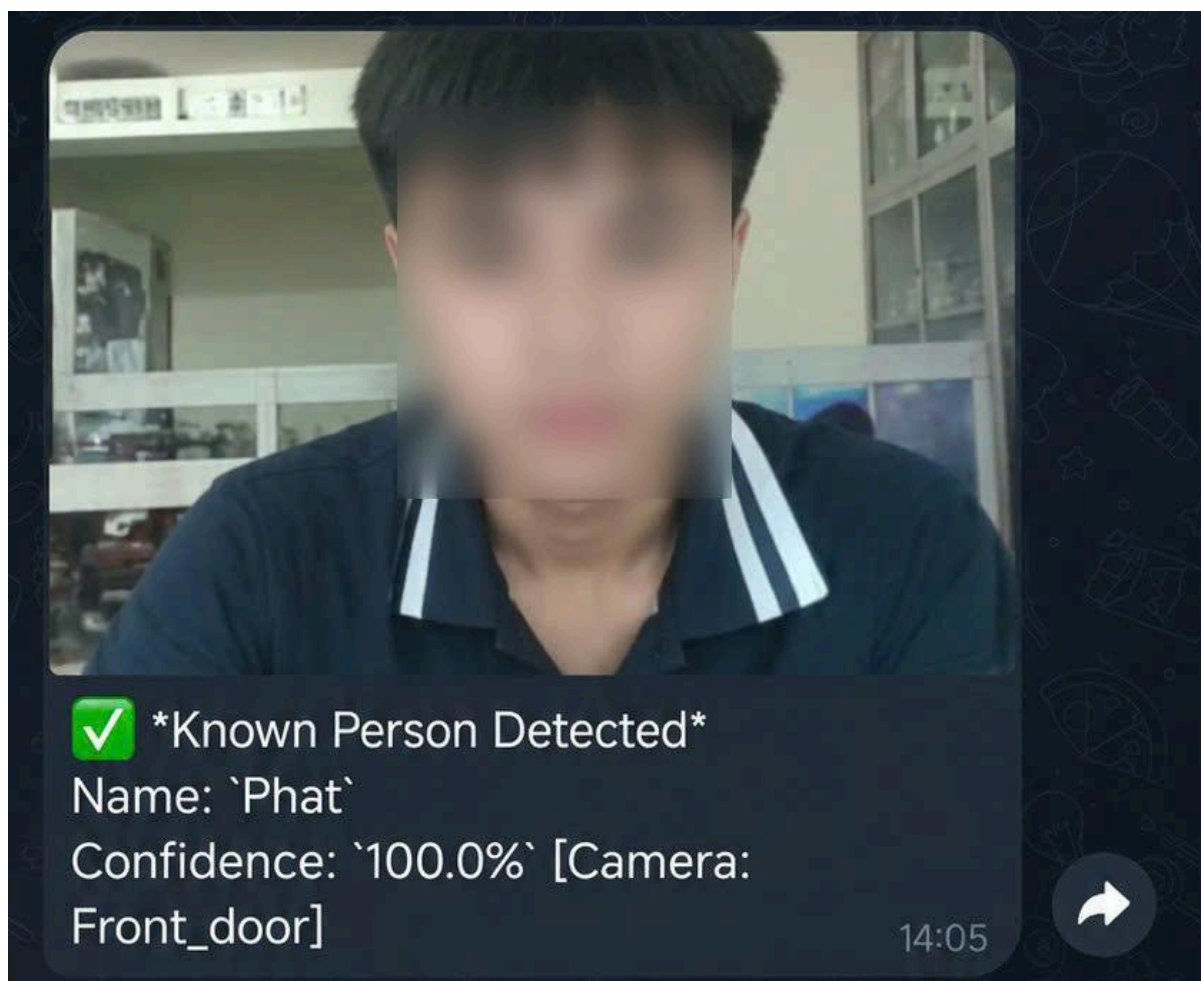
Phương pháp AI_METHOD = 'aws' cho độ chính xác vượt trội trong các tình huống phức tạp, bao gồm góc quay đa dạng, điều kiện ánh sáng thay đổi và người đứng ở khoảng cách xa. AWS Rekognition phù hợp cho môi trường yêu cầu độ chính xác cao và ít chấp nhận cảnh báo giả.

Ngưỡng confidence threshold được xác định là yếu tố then chốt ảnh hưởng trực tiếp đến chất lượng nhận diện: tăng ngưỡng giúp giảm cảnh báo giả nhưng có thể bỏ sót một số trường hợp biên, trong khi giảm ngưỡng tăng khả năng phát hiện nhưng dễ kích hoạt nhầm. Cần hiệu chỉnh ngưỡng phù hợp với từng môi trường triển khai cụ thể.

Hệ thống ghi nhận kết quả tốt với nhiều tư thế khác nhau, bao gồm cả trường hợp nhận diện người đứng ở khoảng cách xa trong khung hình.

4.2.3 Kết quả gửi cảnh báo Telegram

Telegram Bot hoạt động ổn định và gửi cảnh báo thành công mỗi khi hệ thống phát hiện chuyển động hoặc nhận diện khuôn mặt. Nội dung cảnh báo bao gồm thời gian phát hiện, loại sự kiện (người quen hoặc người lạ) và ảnh chụp từ ESP32-CAM.



Hình 21: Kết quả gửi qua Telegram khi nhận diện người quen



Hình 22: Kết quả gửi qua Telegram khi nhận diện người lạ

Kết quả cho thấy thông báo được gửi thành công đến Telegram kèm hình ảnh và thông tin nhận diện. Kết quả thử nghiệm cho thấy cảnh báo được gửi nhanh, nội dung hiển thị đầy đủ và không xuất hiện tình trạng gửi trùng lặp.

4.2.4 Hiệu suất và độ trễ xử lý

Chỉ số	Giá trị đo được
Thời gian từ PIR đến nhận Telegram	1 – 1,5 giây trên mạng LAN ổn định
Thời gian backend xử lý mỗi sự kiện	0,2 – 0,8 giây trên máy tính phổ thông

Độ trễ tổng thể dưới 2 giây đáp ứng tốt yêu cầu cảnh báo theo thời gian thực cho hệ thống giám sát quy mô nhỏ và vừa.

4.2.5 Đánh giá tổng thể

Hệ thống phù hợp cho các ứng dụng giám sát nhà riêng, văn phòng nhỏ và khu vực cần kiểm soát ra vào, nhờ khả năng phát hiện nhanh, gửi ảnh trực tiếp và tích hợp thông báo tức thì qua Telegram. Kiến trúc đa camera kết hợp MQTT và coordinator cho phép mở rộng số lượng camera mà không cần thay đổi cấu trúc hệ thống, đồng thời giảm thiểu đáng kể nguy cơ mất thông tin khi nhiều điểm giám sát hoạt động song song. Cảnh báo giả hầu như không xảy ra trong quá trình thử nghiệm nhờ cơ chế debounce, cooldown và logic lọc duplicate của coordinator. Thiết kế backend rõ ràng, module hóa tốt, phù hợp cho mục đích học tập, nghiên cứu và triển khai thực tế ở quy mô nhỏ.

PHẦN V. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết luận

Đồ án được thực hiện bởi nhóm 5 đã xây dựng thành công một hệ thống an ninh thông minh + Chống trộm với hệ thống này có thể giám sát theo thời gian thực. Bên cạnh những tính năng cơ bản còn tích hợp thêm những tính năng nâng cao như Multi camera, AWS Rekognition. Hệ thống cho thấy khả năng phát hiện chuyển động, chụp ảnh và gửi cảnh báo Telegram nhanh chóng, ổn định.

Kiến trúc phân lớp rõ ràng giúp dễ mở rộng, dễ hiệu chỉnh và phù hợp cho đồ án học tập có thể triển khai trong nhà ở, phòng trọ. Việc tích hợp AI giúp lọc bớt cảnh báo giả và nâng cao tính thông minh hơn so với hệ thống chỉ dùng cảm biến PIR.

Qua quá trình thử nghiệm thực tế, hệ thống đạt độ trễ phản hồi trung bình dưới 2 giây từ lúc PIR phát hiện chuyển động đến khi người dùng nhận cảnh báo trên Telegram. AWS Rekognition nhận diện khuôn mặt với confidence score trên 99.99% trong điều kiện ánh sáng tốt. Tổng chi phí triển khai phần cứng chỉ khoảng 689.000 VNĐ, cho thấy tính khả thi cao khi ứng dụng thực tế trong hộ gia đình và văn phòng nhỏ.

5.2 Hướng phát triển trong tương lai

Mở rộng thành hệ thống đa cảm biến, đồng bộ nhận diện theo khu vực để nâng cao độ phủ và độ tin cậy. Xây dựng dashboard web hiển thị trạng thái camera, lịch sử cảnh báo, hình ảnh sự kiện và biểu đồ phân tích.

Nâng cấp mô-đun AI bằng các mô hình hiện đại hơn (YOLO, MobileNet, Detectron, v.v.) để cải thiện độ chính xác và giảm false positive/negative. Phát triển chức năng phân loại sự kiện nâng cao: phân biệt người, vật nuôi, phương tiện, hoặc tình huống khẩn cấp, từ đó đưa ra cảnh báo phù hợp.

Ứng dụng vào thực tế và độ phủ sóng tương lai:

Giám sát cửa ra vào, hành lang, phòng kho, văn phòng và nhà riêng. Cảnh báo xâm nhập tại cửa hàng, quán cà phê, cửa ngách hoặc khu vực không phép. Hệ thống an ninh cho homestay, villa, căn hộ cho thuê khi cần thông báo nhanh đến chủ nhà. Giám sát khu vực công trình, kho vật tư, phòng server khi cần thông tin từ xa. Phối hợp nhiều camera để giám sát khu vực rộng, giảm báo động giả và nâng cao độ phủ.

THAM KHẢO

- [1] "ESP32-CAM AI-Thinker – Tìm hiểu board ESP32-CAM," *mecsu.vn*. [Online]. Available: <https://mecsu.vn/ho-tro-ky-thuat/tim-hieu-esp32cam-aithinker-board-la-gi.5Q1>
- [2] "Module còi chip 3.3V-5V," *dientu360.com*. [Online]. Available: <https://dientu360.com/module-coi-chip>
- [3] "Cảm biến thân nhiệt chuyển động PIR HC-SR501," *nshopvn.com*. [Online]. Available: <https://nshopvn.com/product/cam-bien-than-nhiet-chuyen-dong-pir-hc-sr501/>
- [4] "Breadboard cảm linh kiện 400 lỗ," *thegioiic.com*. [Online]. Available: <https://www.thegioiic.com/breadboard-cam-linh-kien-400-lo-trong-suot>
- [5] "Dây cắm test board đực – cái 30cm," *hshop.vn*. [Online]. Available: <https://hshop.vn/day-camtest-board-duc-coi30cm40soi>
- [6] "Động cơ RC Servo 9G 360°," *hshop.vn*. [Online]. Available: <https://hshop.vn/dong-co-rc-servo-9g-360>
- [7] Canonical Ltd., "Ubuntu Desktop," *ubuntu.com*. [Online]. Available: <https://ubuntu.com/download/desktop>
- [8] EMQX, "MQTTX – MQTT Client Tool," *mqttx.app*. [Online]. Available: <https://mqttx.app/>
- [9] Arduino, "Arduino IDE," *arduino.cc*. [Online]. Available: <https://www.arduino.cc/>
- [10] Python Software Foundation, "Python Programming Language," *python.org*. [Online]. Available: <https://www.python.org/>
- [11] Telegram, "Telegram Bot API," *core.telegram.org*. [Online]. Available: <https://core.telegram.org/bots/api>
- [12] GitHub Inc., "GitHub," *github.com*. [Online]. Available: <https://github.com/>
- [13] Amazon Web Services, "Amazon Rekognition," *aws.amazon.com*. [Online]. Available: <https://aws.amazon.com/rekognition/>

Link github:

<https://github.com/NhutKhanh24520782/Smart-Security-System-with-ESP32-CAM-and-AI-Detection/tree/main>